Roteiro de estudo
Márcio Fantini
Coltec-UFMG

MySQL: junções

Consultas envolvendo múltiplas tabelas

Relacionamentos, chaves e junções internas e externas

Continuação do roteiro de MySQL básico

Sumário

1 Apresentação e roteiro de aulas **1**

2 Por que dividir os dados em tabelas? **2**
2.1 Chave primária . . . 2
2.2 Chave estrangeira . . . 2
2.3 Cardinalidade: um para muitos . . . 3

3 Criando o banco de dados **3**
3.1 Banco e tabela clientes . . . 3
3.2 Tabela produtos . . . 3
3.3 Tabela vendas e chaves estrangeiras . . . 4
3.4 Clientes cadastrados . . . 5
3.5 Produtos cadastrados . . . 5
3.6 Vendas realizadas . . . 6
3.7 Comandos iniciais de conferência . . . 6

4 Entendendo os relacionamentos na prática **7**
4.1 Aliases: apelidos para as tabelas . . . 7

5 INNER JOIN: somente linhas correspondentes **8**
5.1 Clientes e vendas . . . 8
5.2 Vendas e produtos . . . 8
5.3 INNER JOIN com WHERE . . . 9

6 Combinando as três tabelas **9**
6.1 Filtrando informações de tabelas diferentes . . . 10
6.2 Comparando o preço atual com o preço histórico . . . 10

7 LEFT JOIN: preservando a tabela da esquerda **11**
7.1 Todos os clientes, tenham comprado ou não . . . 11
7.2 Encontrando clientes que nunca compraram . . . 12
7.3 Todos os produtos, tenham sido vendidos ou não . . . 12
7.4 LEFT JOIN envolvendo três tabelas . . . 13

8 RIGHT JOIN: preservando a tabela da direita **13**
8.1 Todas as linhas de clientes . . . 13
8.2 Todos os produtos, vendidos ou não . . . 14

8.3 Equivalência entre LEFT e RIGHT 14

9 Junção completa: FULL OUTER JOIN 15

9.1 Simulação com LEFT JOIN, RIGHT JOIN e UNION 15
9.2 Versão explícita com UNION ALL 16
9.3 O efeito das chaves estrangeiras neste banco 16
9.4 Junção completa entre clientes e produtos por identificador 17

10 Comparando os tipos de JOIN 18

11 Junções com agregações e GROUP BY 19

11.1 Quantidade de compras por cliente 19
11.2 Incluindo clientes sem compra 19
11.3 Total gasto por cliente 20
11.4 Unidades vendidas por produto 20
11.5 Faturamento por estado do cliente 21
11.6 Faturamento por categoria de produto 21
11.7 Filtrando grupos com HAVING 21

12 Erros frequentes e como evitá-los 22

12.1 Erro 1: esquecer a condição de relacionamento 22
12.2 Erro 2: não qualificar nomes de colunas 22
12.3 Erro 3: transformar LEFT JOIN em INNER JOIN 23
12.4 Erro 4: usar COUNT(*) quando se deseja zero 23
12.5 Erro 5: somar preços atuais em vez dos valores históricos 24
12.6 Erro 6: confundir uma venda com um cliente 24

13 Revisão geral: estudo dirigido 24

14 Gabarito sugerido 26

14.1 Atividade 4.1 26
14.2 Atividade 5.1 26
14.3 Atividade 6.1 26
14.4 Atividade 7.1 27
14.5 Atividade 8.1 27
14.6 Atividade 9.1 28
14.7 Atividade 10.1 29
14.8 Atividade 11.1 29
14.9 Atividade 12.1 30

14.10Revisão 13.2 — INNER JOIN 30
14.11Revisão 13.3 — junções externas 31
14.12Revisão 13.4 — relatórios gerenciais 31

15 Checklist do estudante **32**

1 Apresentação e roteiro de aulas

No roteiro anterior, cada consulta acessava somente uma tabela. Agora vamos avançar para perguntas que dependem de dados armazenados em tabelas diferentes:

- Qual cliente realizou cada venda?
- Qual produto foi comprado e qual foi o valor total?
- Existem clientes cadastrados que ainda não fizeram compras?
- Existem produtos cadastrados que nunca foram vendidos?
- Quanto cada cliente gastou, incluindo quem gastou zero?

Objetivos de aprendizagem

Ao final deste roteiro, o estudante deverá conseguir:

- identificar chaves primárias e chaves estrangeiras;
- interpretar relacionamentos de um para muitos;
- construir junções com `INNER JOIN` e `ON`;
- preservar linhas com `LEFT JOIN` e `RIGHT JOIN`;
- reconhecer valores `NULL` gerados por junções externas;
- combinar três tabelas na mesma consulta;
- agregar resultados de junções com `COUNT`, `SUM` e `GROUP BY`;
- simular `FULL OUTER JOIN` no MySQL com `UNION`;
- evitar erros comuns envolvendo duplicação de linhas, filtros e contagens.

Atenção

Este material não utiliza subconsultas. A expressão `UNION`, apresentada na junção completa, apenas reúne os resultados de duas consultas e não deve ser confundida com uma subconsulta.

Etapa	Conteúdo	Prática
1	Modelo relacional, chaves, cardinalidade e criação do banco	Seções 2 a 4
2	Junções internas entre duas e três tabelas	Seções 5 e 6
3	Junções externas à esquerda e à direita	Seções 7 e 8
4	Junção completa, `UNION` e comparação entre resultados	Seção 9
5	Filtros, agregação e armadilhas frequentes	Seções 10 a 12
6	Estudo dirigido, desafios e correção comentada	Seções 13 e 14

2 Por que dividir os dados em tabelas?

Imagine uma tabela única contendo, em cada venda, o nome do cliente, a cidade, o nome do produto, sua categoria e seu preço. Se o mesmo cliente comprar dez vezes, seus dados cadastrais seriam repetidos dez vezes.

Essa repetição causa pelo menos três problemas:

1. desperdício de armazenamento;
2. maior possibilidade de inconsistências;
3. necessidade de alterar muitas linhas quando um cadastro muda.

Uma organização mais adequada separa os dados em tabelas relacionadas:

Tabela	O que ela representa
`clientes`	pessoas cadastradas na loja;
`produtos`	itens disponíveis para comercialização;
`vendas`	operações de compra, cada uma indicando um cliente e um produto.

Ideia fundamental

As tabelas são armazenadas separadamente, mas podem ser combinadas durante uma consulta por meio de colunas que representam o relacionamento entre elas. Essa combinação é chamada **junção**, ou **join**.

2.1 Chave primária

Uma **chave primária** identifica uma linha de forma única.

Código 1: Exemplos de chaves primárias

```
clientes.id_cliente
produtos.id_produto
vendas.id_venda
```

Dois clientes podem ter o mesmo nome, mas não podem possuir o mesmo `id_cliente`. É o identificador, e não o nome, que deve ser usado para relacionar linhas com segurança.

2.2 Chave estrangeira

Uma **chave estrangeira** guarda o identificador de uma linha localizada em outra tabela.

Código 2: Chaves estrangeiras presentes em vendas

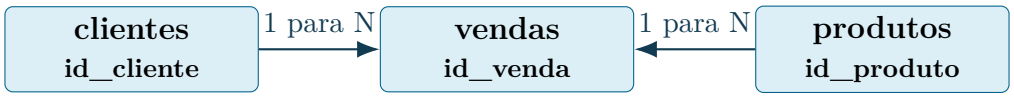
```
vendas.id_cliente  --> clientes.id_cliente
vendas.id_produto  --> produtos.id_produto
```

Integridade referencial

Ao declarar uma chave estrangeira, o banco impede, por exemplo, que uma venda aponte para um cliente ou produto inexistente. Isso protege a consistência do modelo relacional.

2.3 Cardinalidade: um para muitos

Um cliente pode aparecer em várias vendas. Um produto também pode aparecer em várias vendas. Por isso, temos dois relacionamentos do tipo **um para muitos**.



Como interpretar o diagrama

Cada venda possui um cliente e um produto. Entretanto, um mesmo cliente pode participar de várias vendas, e um mesmo produto pode ser vendido várias vezes.

3 Criando o banco de dados

O banco `loja_juncoes` é independente do banco criado no roteiro anterior. Assim, os dois materiais podem ser usados separadamente.

3.1 Banco e tabela clientes

Código 3: Criação do banco e da tabela clientes

```sql
CREATE DATABASE loja_juncoes
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE loja_juncoes;

CREATE TABLE clientes (
    id_cliente     INT PRIMARY KEY AUTO_INCREMENT,
    nome           VARCHAR(100) NOT NULL,
    cidade         VARCHAR(60) NOT NULL,
    estado         CHAR(2) NOT NULL,
    data_cadastro  DATE NOT NULL,
    ativo          BOOLEAN NOT NULL DEFAULT TRUE
);
```

3.2 Tabela produtos

Código 4: Estrutura da tabela produtos

```sql
CREATE TABLE produtos (
    id_produto      INT PRIMARY KEY AUTO_INCREMENT,
    nome_produto    VARCHAR(100) NOT NULL,
    categoria       VARCHAR(50) NOT NULL,
    preco           DECIMAL(10,2) NOT NULL,
    estoque         INT NOT NULL,
    ativo           BOOLEAN NOT NULL DEFAULT TRUE
);
```

3.3 Tabela vendas e chaves estrangeiras

Código 5: Estrutura da tabela vendas

```sql
CREATE TABLE vendas (
    id_venda          INT PRIMARY KEY AUTO_INCREMENT,
    id_cliente        INT NOT NULL,
    id_produto        INT NOT NULL,
    quantidade        INT NOT NULL,
    valor_unitario    DECIMAL(10,2) NOT NULL,
    data_venda        DATE NOT NULL,
    forma_pagamento   VARCHAR(30) NOT NULL,

    CONSTRAINT fk_vendas_clientes
      FOREIGN KEY (id_cliente)
      REFERENCES clientes (id_cliente),

    CONSTRAINT fk_vendas_produtos
      FOREIGN KEY (id_produto)
      REFERENCES produtos (id_produto)
);
```

Por que registrar valor_unitario em vendas?

O preço atual do produto pode mudar. O campo `vendas.valor_unitario` informa o valor praticado no momento da venda. Assim, `produtos.preco` é o preço registrado na loja (como se fosse o pre ç de um catálogo. Podemos ainda ter um campo que registra o preço quando ele foi adquirido. O valor `vendas.valor_unitario` registra o preço da venda (que pode mudar a cada venda (para mais ou para menos)).

3.4 Clientes cadastrados

Código 6: Inserindo 12 clientes

```sql
INSERT INTO clientes
  (nome, cidade, estado, data_cadastro, ativo)
VALUES
('Ana Souza',       'Belo Horizonte', 'MG', '2025-01-03', TRUE),
('Bruno Lima',      'Contagem',       'MG', '2025-01-08', TRUE),
('Carla Mendes',    'São Paulo',      'SP', '2025-01-12', TRUE),
('Daniel Rocha',    'Rio de Janeiro', 'RJ', '2025-01-20', TRUE),
('Elisa Martins',   'Belo Horizonte', 'MG', '2025-02-01', TRUE),
('Fábio Costa',     'Salvador',       'BA', '2025-02-09', TRUE),
('Gabriela Alves',  'Curitiba',       'PR', '2025-02-15', FALSE),
('Henrique Nunes',  'São Paulo',      'SP', '2025-03-02', TRUE),
('Isabela Freitas', 'Vitória',        'ES', '2025-03-10', TRUE),
('João Ribeiro',    'Belo Horizonte', 'MG', '2025-03-22', TRUE),
('Karen Oliveira',  'Recife',         'PE', '2025-04-05', TRUE),
('Lucas Pereira',   'Contagem',       'MG', '2025-04-18', TRUE);
```

3.5 Produtos cadastrados

Código 7: Inserindo 12 produtos

```sql
INSERT INTO produtos
  (nome_produto, categoria, preco, estoque, ativo)
VALUES
('Notebook Pro 14',      'Informática', 4590.00,  8, TRUE),
('Mouse sem fio',        'Acessórios',   120.00, 35, TRUE),
('Monitor 24',           'Informática',  980.00, 12, TRUE),
('Teclado mecânico',     'Acessórios',   350.00, 20, TRUE),
('Smartphone X',         'Telefonia',   2850.00, 10, TRUE),
('Webcam HD',            'Acessórios',   210.00, 18, TRUE),
('Tablet 10',            'Informática', 1750.00,  9, TRUE),
('Fone Bluetooth',       'Telefonia',    190.00, 25, TRUE),
('SSD 1 TB',             'Informática',  520.00, 14, TRUE),
('Cadeira ergonômica',   'Escritório',  1350.00,  6, TRUE),
('Impressora térmica',   'Escritório',   780.00,  5, TRUE),
('Projetor multimídia',  'Informática', 2390.00,  4, TRUE);
```

3.6 Vendas realizadas

Código 8: Inserindo 18 vendas

```sql
INSERT INTO vendas
  (id_cliente, id_produto, quantidade, valor_unitario,
   data_venda, forma_pagamento)
VALUES
(1,  1, 1, 4590.00, '2025-01-08', 'Cartão'),
(2,  2, 2,  120.00, '2025-01-10', 'Pix'),
(3,  3, 1,  980.00, '2025-01-15', 'Cartão'),
(5,  4, 1,  350.00, '2025-02-03', 'Pix'),
(1,  2, 1,  120.00, '2025-02-12', 'Dinheiro'),
(6,  5, 1, 2850.00, '2025-02-21', 'Cartão'),
(8,  6, 2,  210.00, '2025-03-02', 'Pix'),
(9,  7, 1, 1750.00, '2025-03-18', 'Boleto'),
(10, 8, 3,  190.00, '2025-03-25', 'Pix'),
(12, 3, 1,  980.00, '2025-04-07', 'Cartão'),
(2,  4, 1,  350.00, '2025-04-16', 'Dinheiro'),
(3,  5, 1, 2750.00, '2025-04-29', 'Cartão'),
(5,  9, 2,  520.00, '2025-05-06', 'Pix'),
(6,  8, 2,  190.00, '2025-05-20', 'Boleto'),
(8, 10, 1, 1350.00, '2025-06-04', 'Cartão'),
(9,  2, 3,  115.00, '2025-06-18', 'Pix'),
(10, 9, 1,  520.00, '2025-07-01', 'Boleto'),
(1,  6, 1,  210.00, '2025-07-12', 'Pix');
```

Atenção

Os clientes **Daniel Rocha**, **Gabriela Alves** e **Karen Oliveira** não possuem vendas. Os produtos **Impressora térmica** e **Projetor multimídia** nunca foram vendidos. Essas ausências são intencionais e permitem comparar os tipos de junção.

3.7 Comandos iniciais de conferência

Código 9: Inspecionando a estrutura e os dados

```sql
SHOW TABLES;

DESCRIBE clientes;
DESCRIBE produtos;
DESCRIBE vendas;

SELECT * FROM clientes;
SELECT * FROM produtos;
SELECT * FROM vendas;
```

4 Entendendo os relacionamentos na prática

Observe uma venda da tabela `vendas`:

id_venda	id_cliente	id_produto	quantidade	valor_unitario
1	1	1	1	4590,00

Sozinha, essa linha não mostra o nome do cliente nem o nome do produto. Porém:

- `id_cliente = 1` aponta para `Ana Souza`;
- `id_produto = 1` aponta para `Notebook Pro 14`.

Uma junção recupera essas informações em uma única resposta.

A condição de junção

A condição `ON` informa quais colunas devem ser comparadas. Neste banco: `vendas.id_cliente = clientes.id_cliente` e `vendas.id_produto = produtos.id_produto`.

4.1 Aliases: apelidos para as tabelas

Usaremos apelidos curtos para deixar os comandos mais legíveis:

Tabela	Alias	Exemplo de coluna
`clientes`	`c`	`c.nome`
`produtos`	`p`	`p.nome_produto`
`vendas`	`v`	`v.data_venda`

Código 10: Apelidos usados no FROM

```
FROM clientes AS c
FROM produtos AS p
FROM vendas AS v
```

A palavra `AS` é opcional para esses apelidos, mas será mantida para tornar a intenção explícita.

Atividade 4.1

Antes de escrever consultas, responda:

a) Qual é a chave primária de cada uma das três tabelas?

b) Quais campos de `vendas` são chaves estrangeiras?

c) Um cliente pode aparecer em mais de uma venda? Por quê?

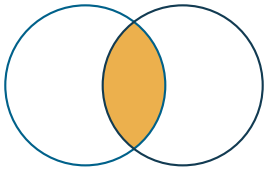
d) Um produto pode aparecer em mais de uma venda? Por quê?

e) Um cliente precisa ter feito uma compra para existir no cadastro?

5 INNER JOIN: somente linhas correspondentes

`INNER JOIN` retorna somente as combinações para as quais existe correspondência nos dois lados da junção.

Somente correspondências



clientes vendas

Atenção

Os diagramas de conjuntos são uma aproximação visual. Na prática, uma linha do cliente pode aparecer várias vezes quando existem várias vendas correspondentes.

5.1 Clientes e vendas

Código 11: Primeira junção interna

```sql
SELECT
    c.nome AS cliente,
    v.id_venda,
    v.data_venda,
    v.quantidade
FROM clientes AS c
INNER JOIN vendas AS v
    ON c.id_cliente = v.id_cliente
ORDER BY c.nome, v.data_venda;
```

Leia o resultado

O resultado contém somente clientes que realizaram vendas. Ana Souza aparece mais de uma vez porque participou de mais de uma compra. Daniel, Gabriela e Karen não aparecem, pois não possuem correspondência em `vendas`.

5.2 Vendas e produtos

Código 12: Identificando os produtos vendidos

```sql
SELECT
```

```
    v.id_venda,
    p.nome_produto,
    p.categoria,
    v.quantidade,
    v.valor_unitario,
    v.quantidade * v.valor_unitario AS total_item
FROM vendas AS v
INNER JOIN produtos AS p
    ON v.id_produto = p.id_produto
ORDER BY v.id_venda;
```

5.3 INNER JOIN com WHERE

Código 13: Compras realizadas por clientes de Minas Gerais

```
SELECT
    c.nome,
    c.cidade,
    v.id_venda,
    v.data_venda
FROM clientes AS c
INNER JOIN vendas AS v
    ON c.id_cliente = v.id_cliente
WHERE c.estado = 'MG'
ORDER BY v.data_venda;
```

Papéis diferentes de ON e WHERE

`ON` descreve como as tabelas se relacionam. `WHERE` seleciona quais linhas do resultado devem permanecer. Em junções externas, essa diferença é especialmente importante.

Atividade 5.1

Escreva consultas com `INNER JOIN` para:

a) exibir o nome do cliente e a data de cada venda;

b) mostrar o nome do produto e a quantidade vendida;

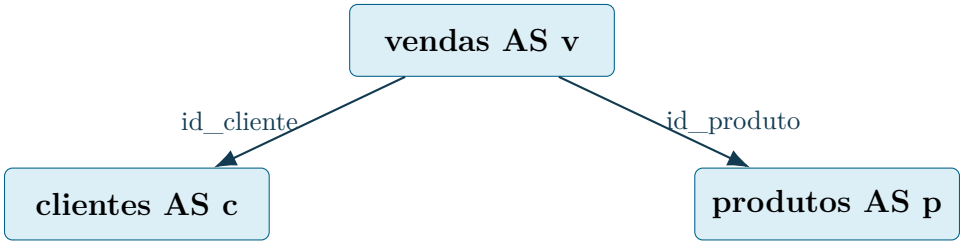
c) listar vendas de produtos da categoria Informática;

d) exibir clientes e vendas pagas por Pix;

e) mostrar os produtos vendidos por valor unitário acima de 500.

6 Combinando as três tabelas

Para reunir cliente, produto e venda na mesma resposta, fazemos duas junções. A tabela `vendas` funciona como ponto de ligação entre as outras duas.



Código 14: Consulta completa com três tabelas

```sql
SELECT
    v.id_venda,
    c.nome AS cliente,
    p.nome_produto AS produto,
    p.categoria,
    v.quantidade,
    v.valor_unitario,
    v.quantidade * v.valor_unitario AS total_venda,
    v.data_venda
FROM vendas AS v
INNER JOIN clientes AS c
    ON v.id_cliente = c.id_cliente
INNER JOIN produtos AS p
    ON v.id_produto = p.id_produto
ORDER BY v.data_venda, v.id_venda;
```

6.1 Filtrando informações de tabelas diferentes

Código 15: Clientes mineiros comprando Informática

```sql
SELECT
    c.nome AS cliente,
    c.cidade,
    p.nome_produto,
    v.data_venda,
    v.quantidade * v.valor_unitario AS total
FROM vendas AS v
INNER JOIN clientes AS c
    ON v.id_cliente = c.id_cliente
INNER JOIN produtos AS p
    ON v.id_produto = p.id_produto
WHERE c.estado = 'MG'
  AND p.categoria = 'Informática'
ORDER BY total DESC;
```

6.2 Comparando o preço atual com o preço histórico

Código 16: Mudanças entre o preço vendido e o preço atual

```sql
SELECT
```

```
    p.nome_produto,
    p.preco AS preco_atual,
    v.valor_unitario AS preco_na_venda,
    p.preco - v.valor_unitario AS diferenca,
    v.data_venda
FROM vendas AS v
INNER JOIN produtos AS p
    ON v.id_produto = p.id_produto
WHERE p.preco <> v.valor_unitario
ORDER BY v.data_venda;
```

Atividade 6.1

Use as três tabelas para:

a) mostrar cliente, produto e data de cada compra;

b) listar cliente, cidade, produto, quantidade e total da venda;

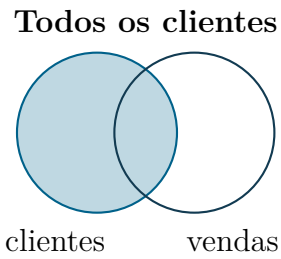
c) mostrar compras da categoria Telefonia pagas com Cartão;

d) listar produtos comprados por clientes de São Paulo;

e) encontrar compras realizadas em maio de 2025, mostrando cliente e produto.

7 LEFT JOIN: preservando a tabela da esquerda

`LEFT JOIN` mantém todas as linhas da tabela escrita antes do `LEFT JOIN`. Quando não existe correspondência no lado direito, as colunas da tabela direita aparecem como `NULL`.



7.1 Todos os clientes, tenham comprado ou não

Código 17: Clientes com e sem vendas

```
SELECT
    c.nome AS cliente,
    v.id_venda,
    v.data_venda,
    v.forma_pagamento
FROM clientes AS c
LEFT JOIN vendas AS v
    ON c.id_cliente = v.id_cliente
```

```sql
ORDER BY c.nome, v.data_venda;
```

Leia o resultado

Daniel, Gabriela e Karen aparecem no resultado. Para esses clientes, `id_venda`, `data_venda` e `forma_pagamento` aparecem como `NULL`, pois não há venda correspondente.

7.2 Encontrando clientes que nunca compraram

Código 18: Filtro de ausência com LEFT JOIN e IS NULL

```sql
SELECT
    c.id_cliente,
    c.nome,
    c.cidade
FROM clientes AS c
LEFT JOIN vendas AS v
    ON c.id_cliente = v.id_cliente
WHERE v.id_venda IS NULL
ORDER BY c.nome;
```

Padrão de ausência

A combinação `LEFT JOIN` + `WHERE tabela_direita.id IS NULL` é um padrão frequente para localizar registros sem correspondência.

7.3 Todos os produtos, tenham sido vendidos ou não

Código 19: Produtos e possíveis vendas

```sql
SELECT
    p.nome_produto,
    p.categoria,
    v.id_venda,
    v.quantidade
FROM produtos AS p
LEFT JOIN vendas AS v
    ON p.id_produto = v.id_produto
ORDER BY p.nome_produto, v.id_venda;
```

Código 20: Produtos nunca vendidos

```sql
SELECT
    p.nome_produto,
    p.categoria,
    p.estoque
FROM produtos AS p
LEFT JOIN vendas AS v
    ON p.id_produto = v.id_produto
```

```
WHERE v.id_venda IS NULL
ORDER BY p.nome_produto;
```

7.4 LEFT JOIN envolvendo três tabelas

Código 21: Todos os clientes e, quando existirem, seus produtos

```
SELECT
    c.nome AS cliente,
    v.id_venda,
    p.nome_produto AS produto,
    v.data_venda
FROM clientes AS c
LEFT JOIN vendas AS v
    ON c.id_cliente = v.id_cliente
LEFT JOIN produtos AS p
    ON v.id_produto = p.id_produto
ORDER BY c.nome, v.data_venda;
```

Atividade 7.1

Utilize `LEFT JOIN` para:

a) listar todos os clientes e suas vendas, inclusive clientes sem compra;

b) mostrar somente os clientes que nunca compraram;

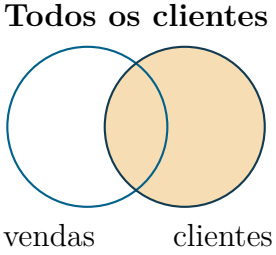
c) listar todos os produtos e as respectivas vendas;

d) mostrar somente os produtos nunca vendidos;

e) listar todos os clientes e, quando houver compra, o nome do produto.

8 RIGHT JOIN: preservando a tabela da direita

`RIGHT JOIN` mantém todas as linhas da tabela colocada à direita. Quando o lado esquerdo não possui correspondência, suas colunas aparecem como `NULL`.



8.1 Todas as linhas de clientes

Código 22: RIGHT JOIN preservando clientes

```sql
SELECT
    c.nome AS cliente,
    v.id_venda,
    v.data_venda
FROM vendas AS v
RIGHT JOIN clientes AS c
    ON v.id_cliente = c.id_cliente
ORDER BY c.nome, v.data_venda;
```

Nesse exemplo, `clientes` está à direita. Portanto, todos os clientes aparecem, inclusive Daniel, Gabriela e Karen.

8.2 Todos os produtos, vendidos ou não

Código 23: RIGHT JOIN preservando produtos

```sql
SELECT
    p.nome_produto,
    p.categoria,
    v.id_venda,
    v.quantidade
FROM vendas AS v
RIGHT JOIN produtos AS p
    ON v.id_produto = p.id_produto
ORDER BY p.nome_produto;
```

8.3 Equivalência entre LEFT e RIGHT

As consultas seguintes preservam o mesmo conjunto de linhas:

Código 24: Forma com LEFT JOIN

```sql
SELECT c.nome, v.id_venda
FROM clientes AS c
LEFT JOIN vendas AS v
    ON c.id_cliente = v.id_cliente;
```

Código 25: Forma equivalente com RIGHT JOIN

```sql
SELECT c.nome, v.id_venda
FROM vendas AS v
RIGHT JOIN clientes AS c
    ON v.id_cliente = c.id_cliente;
```

Escolhendo o lado preservado

`LEFT JOIN` e `RIGHT JOIN` podem ser convertidos um no outro invertendo a posição das tabelas. Na prática, muitos profissionais preferem `LEFT JOIN`, porque a leitura costuma começar pela entidade principal.

Atividade 8.1

Resolva com `RIGHT JOIN`:

a) mostrar todos os clientes e as vendas correspondentes;

b) mostrar todos os produtos e as vendas correspondentes;

c) identificar clientes sem compra;

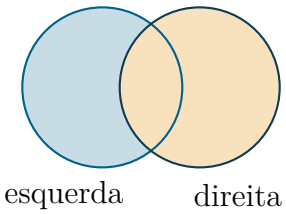
d) identificar produtos sem venda;

e) reescrever o item b usando `LEFT JOIN`.

9 Junção completa: FULL OUTER JOIN

Conceitualmente, uma **junção completa** preserva todas as linhas dos dois lados: linhas correspondentes, linhas exclusivas da esquerda e linhas exclusivas da direita.

Todas as linhas dos dois lados



Atenção

O MySQL **não implementa diretamente a sintaxe** `FULL OUTER JOIN`. Escrever `FROM clientes FULL OUTER JOIN vendas ON ...` produz erro de sintaxe. Para obter o efeito de uma junção completa, combinamos `LEFT JOIN`, `RIGHT JOIN` e `UNION`.

9.1 Simulação com LEFT JOIN, RIGHT JOIN e UNION

Código 26: Simulando FULL OUTER JOIN no MySQL

```sql
SELECT
    c.id_cliente,
    c.nome AS cliente,
    v.id_venda,
    v.data_venda
FROM clientes AS c
LEFT JOIN vendas AS v
    ON c.id_cliente = v.id_cliente

UNION

SELECT
    c.id_cliente,
```

```
    c.nome AS cliente,
    v.id_venda,
    v.data_venda
FROM clientes AS c
RIGHT JOIN vendas AS v
    ON c.id_cliente = v.id_cliente;
```

O papel de UNION

`UNION` junta os resultados das duas consultas e elimina linhas duplicadas. As linhas encontradas pelos dois lados aparecem somente uma vez no resultado final.

9.2 Versão explícita com UNION ALL

Código 27: Junção completa sem duplicar correspondências

```
SELECT
    c.id_cliente,
    c.nome AS cliente,
    v.id_venda,
    v.data_venda
FROM clientes AS c
LEFT JOIN vendas AS v
    ON c.id_cliente = v.id_cliente

UNION ALL

SELECT
    c.id_cliente,
    c.nome AS cliente,
    v.id_venda,
    v.data_venda
FROM clientes AS c
RIGHT JOIN vendas AS v
    ON c.id_cliente = v.id_cliente
WHERE c.id_cliente IS NULL;
```

Leia o resultado

A primeira consulta inclui todas as linhas do lado esquerdo. A segunda adiciona apenas as linhas exclusivas do lado direito. Por isso, `UNION ALL` não duplica as correspondências já incluídas.

9.3 O efeito das chaves estrangeiras neste banco

Neste modelo, `vendas.id_cliente` é obrigatório e referencia um cliente existente. Portanto, não pode haver vendas sem cliente.

Consequentemente, uma junção completa entre `clientes` e `vendas` produz, com esses dados, o mesmo conjunto de linhas do `LEFT JOIN` que preserva todos os clientes.

Atenção

Isso não significa que `FULL OUTER JOIN` e `LEFT JOIN` sejam conceitualmente iguais. O resultado coincide aqui porque a integridade referencial impede registros exclusivos no lado de `vendas`.

9.4 Junção completa entre clientes e produtos por identificador

Para observar linhas exclusivas dos dois lados sem criar uma quarta tabela, podemos usar uma condição **apenas demonstrativa** entre os identificadores dos clientes e dos produtos cadastrados.

Atenção

A comparação `clientes.id_cliente = produtos.id_produto` não representa um relacionamento de negócio. Ela serve somente para visualizar a mecânica da junção completa. Em aplicações reais, as tabelas devem ser combinadas por chaves que possuam significado no modelo.

Código 28: FULL OUTER JOIN conceitual entre conjuntos filtrados

```sql
SELECT
    c.id_cliente,
    c.nome,
    p.id_produto,
    p.nome_produto
FROM clientes AS c
LEFT JOIN produtos AS p
    ON c.id_cliente = p.id_produto
   AND p.id_produto BETWEEN 5 AND 10
WHERE c.id_cliente BETWEEN 1 AND 8

UNION

SELECT
    c.id_cliente,
    c.nome,
    p.id_produto,
    p.nome_produto
FROM clientes AS c
RIGHT JOIN produtos AS p
    ON c.id_cliente = p.id_produto
   AND c.id_cliente BETWEEN 1 AND 8
WHERE p.id_produto BETWEEN 5 AND 10;
```

Leia o resultado

Os clientes 1 a 4 aparecem somente à esquerda; os identificadores 5 a 8 têm correspondência; os produtos 9 e 10 aparecem somente à direita. É um exemplo artificial, mas útil para visualizar os três tipos de ocorrência.

Atividade 9.1

Responda e pratique:

a) Por que `FULL OUTER JOIN` não pode ser executado diretamente no MySQL?

b) Simule uma junção completa entre produtos e vendas usando `UNION`.

c) Reescreva a consulta anterior usando `UNION ALL`.

d) Explique por que, neste banco, não existem vendas sem produto.

e) Identifique a diferença entre `UNION` e `UNION ALL`.

10 Comparando os tipos de JOIN

Tipo	O que é preservado	Pergunta típica
`INNER JOIN`	somente linhas com correspondência	Quais clientes realizaram compras?
`LEFT JOIN`	todas as linhas da tabela esquerda	Quais clientes compraram ou ainda não compraram?
`RIGHT JOIN`	todas as linhas da tabela direita	Quais produtos existem, tenham sido vendidos ou não?
`FULL OUTER`	todas as linhas dos dois lados	Quais registros existem em qualquer um dos lados?

Código 29: Comparação direta: INNER JOIN

```
SELECT c.nome, v.id_venda
FROM clientes AS c
INNER JOIN vendas AS v
    ON c.id_cliente = v.id_cliente;
```

Código 30: Comparação direta: LEFT JOIN

```
SELECT c.nome, v.id_venda
FROM clientes AS c
LEFT JOIN vendas AS v
    ON c.id_cliente = v.id_cliente;
```

Código 31: Comparação direta: RIGHT JOIN

```
SELECT c.nome, v.id_venda
FROM clientes AS c
RIGHT JOIN vendas AS v
    ON c.id_cliente = v.id_cliente;
```

Contagem esperada para os dados propostos

Há 18 vendas e 3 clientes sem compras. Assim, o `INNER JOIN` retorna 18 linhas; o `LEFT JOIN` preservando clientes retorna 21 linhas; e o `RIGHT JOIN` preservando vendas retorna 18 linhas, pois todas as vendas possuem cliente válido.

Atividade 10.1

Antes de executar, estime quantas linhas resultarão de:

a) `produtos INNER JOIN vendas`;

b) `produtos LEFT JOIN vendas`;

c) `produtos RIGHT JOIN vendas`;

d) `vendas RIGHT JOIN produtos`.

Depois execute as consultas e compare suas previsões com os resultados.

11 Junções com agregações e GROUP BY

Ao combinar `JOIN` com `GROUP BY`, é possível criar relatórios por cliente, produto, categoria, cidade ou estado.

11.1 Quantidade de compras por cliente

Código 32: Contando somente clientes que compraram

```sql
SELECT
    c.id_cliente,
    c.nome,
    COUNT(v.id_venda) AS total_compras
FROM clientes AS c
INNER JOIN vendas AS v
    ON c.id_cliente = v.id_cliente
GROUP BY c.id_cliente, c.nome
ORDER BY total_compras DESC, c.nome;
```

11.2 Incluindo clientes sem compra

Código 33: Clientes com zero ou mais compras

```sql
SELECT
    c.id_cliente,
    c.nome,
    COUNT(v.id_venda) AS total_compras
FROM clientes AS c
LEFT JOIN vendas AS v
    ON c.id_cliente = v.id_cliente
```

```
GROUP BY c.id_cliente, c.nome
ORDER BY total_compras DESC, c.nome;
```

Atenção

Em um `LEFT JOIN`, `COUNT(*)` contaria também a linha preservada de um cliente sem compra e poderia produzir 1. Para obter zero nesses casos, use `COUNT(v.id_venda)`, porque `COUNT(coluna)` ignora `NULL`.

11.3 Total gasto por cliente

Código 34: Somando as compras com COALESCE

```
SELECT
    c.id_cliente,
    c.nome,
    COUNT(v.id_venda) AS total_compras,
    COALESCE(
        SUM(v.quantidade * v.valor_unitario),
        0
    ) AS total_gasto
FROM clientes AS c
LEFT JOIN vendas AS v
    ON c.id_cliente = v.id_cliente
GROUP BY c.id_cliente, c.nome
ORDER BY total_gasto DESC, c.nome;
```

A função COALESCE

`COALESCE(expressao, 0)` devolve o primeiro argumento não nulo. Quando não há compras, `SUM(...)` resulta em `NULL`; com `COALESCE`, o relatório passa a mostrar zero.

11.4 Unidades vendidas por produto

Código 35: Relatório de vendas por produto

```
SELECT
    p.id_produto,
    p.nome_produto,
    p.categoria,
    COUNT(v.id_venda) AS numero_vendas,
    COALESCE(SUM(v.quantidade), 0) AS unidades_vendidas,
    COALESCE(
        SUM(v.quantidade * v.valor_unitario),
        0
    ) AS faturamento
FROM produtos AS p
LEFT JOIN vendas AS v
    ON p.id_produto = v.id_produto
GROUP BY p.id_produto, p.nome_produto, p.categoria
```

```
ORDER BY faturamento DESC, p.nome_produto;
```

11.5 Faturamento por estado do cliente

Código 36: Resumo de compras por estado

```
SELECT
    c.estado,
    COUNT(v.id_venda) AS numero_vendas,
    SUM(v.quantidade * v.valor_unitario) AS faturamento
FROM vendas AS v
INNER JOIN clientes AS c
    ON v.id_cliente = c.id_cliente
GROUP BY c.estado
ORDER BY faturamento DESC;
```

11.6 Faturamento por categoria de produto

Código 37: Resumo de compras por categoria

```
SELECT
    p.categoria,
    COUNT(v.id_venda) AS numero_vendas,
    COALESCE(SUM(v.quantidade), 0) AS unidades,
    COALESCE(
        SUM(v.quantidade * v.valor_unitario),
        0
    ) AS faturamento
FROM produtos AS p
LEFT JOIN vendas AS v
    ON p.id_produto = v.id_produto
GROUP BY p.categoria
ORDER BY faturamento DESC;
```

11.7 Filtrando grupos com HAVING

Código 38: Clientes que gastaram mais de 1000

```
SELECT
    c.id_cliente,
    c.nome,
    SUM(v.quantidade * v.valor_unitario) AS total_gasto
FROM clientes AS c
INNER JOIN vendas AS v
    ON c.id_cliente = v.id_cliente
GROUP BY c.id_cliente, c.nome
HAVING SUM(v.quantidade * v.valor_unitario) > 1000
ORDER BY total_gasto DESC;
```

Atividade 11.1

Monte relatórios para:

a) contar compras por cliente, incluindo quem nunca comprou;

b) mostrar o total gasto por cada cliente, incluindo valores zero;

c) somar as unidades vendidas de cada produto;

d) calcular o faturamento por categoria;

e) contar compras por cidade do cliente;

f) mostrar clientes com pelo menos duas compras;

g) mostrar produtos com faturamento superior a 1.000.

12 Erros frequentes e como evitá-los

12.1 Erro 1: esquecer a condição de relacionamento

Uma consulta que combina todas as linhas de uma tabela com todas as linhas da outra cria um **produto cartesiano**.

Código 39: Exemplo didático de produto cartesiano

```sql
SELECT c.nome, p.nome_produto
FROM clientes AS c
CROSS JOIN produtos AS p;
```

Como há 12 clientes e 12 produtos, essa consulta produz 144 combinações. Isso não informa quem comprou cada produto.

Conceito-chave

O `CROSS JOIN` é válido quando todas as combinações são desejadas. Porém, para descobrir compras reais, a consulta deve passar pela tabela `vendas` e usar condições de junção apropriadas.

12.2 Erro 2: não qualificar nomes de colunas

Código 40: Usando aliases para esclarecer a origem

```sql
SELECT
    c.id_cliente,
    c.nome,
    v.id_venda,
    v.id_cliente
FROM clientes AS c
INNER JOIN vendas AS v
    ON c.id_cliente = v.id_cliente;
```

Quando uma coluna existe em mais de uma tabela, escreva o alias antes do nome, como `c.id_cliente` ou `v.id_cliente`.

12.3 Erro 3: transformar LEFT JOIN em INNER JOIN

Observe a consulta a seguir:

Código 41: Filtro no WHERE elimina os clientes sem compra

```
SELECT c.nome, v.id_venda, v.forma_pagamento
FROM clientes AS c
LEFT JOIN vendas AS v
    ON c.id_cliente = v.id_cliente
WHERE v.forma_pagamento = 'Pix';
```

Clientes sem compra possuem `v.forma_pagamento = NULL`. Como esse valor não satisfaz a condição, essas linhas são descartadas.

Para preservar todos os clientes e associar apenas vendas pagas por Pix, mova o filtro para a cláusula `ON`:

Código 42: Filtro no ON preserva todos os clientes

```
SELECT c.nome, v.id_venda, v.forma_pagamento
FROM clientes AS c
LEFT JOIN vendas AS v
    ON c.id_cliente = v.id_cliente
   AND v.forma_pagamento = 'Pix'
ORDER BY c.nome;
```

Leia o resultado

A primeira consulta mostra somente clientes com compra por Pix. A segunda mostra todos os clientes; as colunas da venda ficam nulas quando não existe compra por Pix para aquele cliente.

12.4 Erro 4: usar COUNT(*) quando se deseja zero

Código 43: Comparando COUNT(*) com COUNT(v.id_venda)

```
SELECT
    c.nome,
    COUNT(*) AS linhas_resultantes,
    COUNT(v.id_venda) AS compras_reais
FROM clientes AS c
LEFT JOIN vendas AS v
    ON c.id_cliente = v.id_cliente
GROUP BY c.id_cliente, c.nome
ORDER BY c.nome;
```

12.5 Erro 5: somar preços atuais em vez dos valores históricos

Código 44: Faturamento baseado no valor efetivamente vendido

```sql
SELECT
    p.nome_produto,
    SUM(v.quantidade * v.valor_unitario) AS faturamento_real
FROM vendas AS v
INNER JOIN produtos AS p
    ON v.id_produto = p.id_produto
GROUP BY p.id_produto, p.nome_produto;
```

O faturamento deve usar `v.valor_unitario`. O preço `p.preco` pode ter sido alterado depois da venda.

12.6 Erro 6: confundir uma venda com um cliente

Um cliente com três vendas produz três linhas em uma junção. Caso seja necessário listar cada cliente somente uma vez, use `DISTINCT`:

Código 45: Clientes que compraram, sem repetição

```sql
SELECT DISTINCT
    c.id_cliente,
    c.nome
FROM clientes AS c
INNER JOIN vendas AS v
    ON c.id_cliente = v.id_cliente
ORDER BY c.nome;
```

Atividade 12.1

Explique com suas palavras:

a) por que uma junção pode repetir o nome do mesmo cliente;

b) por que `COUNT(*)` pode ser inadequado em `LEFT JOIN`;

c) como um filtro em `WHERE` pode eliminar linhas preservadas;

d) qual preço deve ser usado para calcular faturamento histórico;

e) por que cliente e produto precisam ser ligados por meio de vendas.

13 Revisão geral: estudo dirigido

Resolva as atividades sem consultar o gabarito. Identifique primeiro a entidade que precisa ser preservada e somente depois escolha o tipo de junção.

Atividade 13.1 — relacionamentos

1. Identifique as três chaves primárias do banco.
2. Identifique as duas chaves estrangeiras da tabela vendas.
3. Explique o relacionamento entre clientes e vendas.
4. Explique o relacionamento entre produtos e vendas.
5. Justifique a existência dos campos `preco` e `valor_unitario`.

Atividade 13.2 — INNER JOIN

1. Liste cliente, data e forma de pagamento de todas as compras.
2. Liste produto, categoria, quantidade e total de cada venda.
3. Mostre cliente, cidade, produto e data em uma mesma consulta.
4. Liste compras feitas por clientes de Minas Gerais.
5. Liste compras de Informática pagas por Pix.

Atividade 13.3 — junções externas

1. Liste todos os clientes, tenham realizado compras ou não.
2. Liste somente clientes que nunca compraram.
3. Liste todos os produtos, tenham sido vendidos ou não.
4. Liste somente produtos nunca vendidos.
5. Reescreva uma consulta com `LEFT JOIN` usando `RIGHT JOIN`.
6. Simule uma junção completa entre produtos e vendas.

Atividade 13.4 — relatórios gerenciais

1. Conte as compras de cada cliente, incluindo zero.
2. Calcule o total gasto por cada cliente, incluindo zero.
3. Calcule a quantidade vendida de cada produto.
4. Calcule o faturamento por categoria.
5. Calcule o faturamento por estado do cliente.
6. Liste clientes com pelo menos duas compras.
7. Liste produtos sem vendas e informe o estoque disponível.
8. Mostre, por mês, o faturamento da categoria Informática.

14 Gabarito sugerido

14.1 Atividade 4.1

a) As chaves primárias são `clientes.id_cliente`, `produtos.id_produto` e `vendas.id_venda`.

b) As chaves estrangeiras são `vendas.id_cliente` e `vendas.id_produto`.

c) Sim. Um cliente pode participar de muitas vendas.

d) Sim. Um produto pode estar presente em muitas vendas.

e) Não. O cadastro do cliente existe independentemente de compras.

14.2 Atividade 5.1

```sql
SELECT c.nome, v.data_venda
FROM clientes AS c
INNER JOIN vendas AS v ON c.id_cliente = v.id_cliente;

SELECT p.nome_produto, v.quantidade
FROM vendas AS v
INNER JOIN produtos AS p ON v.id_produto = p.id_produto;

SELECT p.nome_produto, v.id_venda, v.data_venda
FROM vendas AS v
INNER JOIN produtos AS p ON v.id_produto = p.id_produto
WHERE p.categoria = 'Informática';

SELECT c.nome, v.id_venda, v.forma_pagamento
FROM clientes AS c
INNER JOIN vendas AS v ON c.id_cliente = v.id_cliente
WHERE v.forma_pagamento = 'Pix';

SELECT p.nome_produto, v.valor_unitario
FROM vendas AS v
INNER JOIN produtos AS p ON v.id_produto = p.id_produto
WHERE v.valor_unitario > 500;
```

14.3 Atividade 6.1

```sql
SELECT c.nome, p.nome_produto, v.data_venda
FROM vendas AS v
INNER JOIN clientes AS c ON v.id_cliente = c.id_cliente
INNER JOIN produtos AS p ON v.id_produto = p.id_produto;

SELECT c.nome, c.cidade, p.nome_produto, v.quantidade,
       v.quantidade * v.valor_unitario AS total
FROM vendas AS v
```

```sql
INNER JOIN clientes AS c ON v.id_cliente = c.id_cliente
INNER JOIN produtos AS p ON v.id_produto = p.id_produto;

SELECT c.nome, p.nome_produto, v.forma_pagamento
FROM vendas AS v
INNER JOIN clientes AS c ON v.id_cliente = c.id_cliente
INNER JOIN produtos AS p ON v.id_produto = p.id_produto
WHERE p.categoria = 'Telefonia'
  AND v.forma_pagamento = 'Cartão';

SELECT c.nome, p.nome_produto
FROM vendas AS v
INNER JOIN clientes AS c ON v.id_cliente = c.id_cliente
INNER JOIN produtos AS p ON v.id_produto = p.id_produto
WHERE c.estado = 'SP';

SELECT c.nome, p.nome_produto, v.data_venda
FROM vendas AS v
INNER JOIN clientes AS c ON v.id_cliente = c.id_cliente
INNER JOIN produtos AS p ON v.id_produto = p.id_produto
WHERE v.data_venda BETWEEN '2025-05-01' AND '2025-05-31';
```

14.4 Atividade 7.1

```sql
SELECT c.nome, v.id_venda, v.data_venda
FROM clientes AS c
LEFT JOIN vendas AS v ON c.id_cliente = v.id_cliente;

SELECT c.nome
FROM clientes AS c
LEFT JOIN vendas AS v ON c.id_cliente = v.id_cliente
WHERE v.id_venda IS NULL;

SELECT p.nome_produto, v.id_venda, v.quantidade
FROM produtos AS p
LEFT JOIN vendas AS v ON p.id_produto = v.id_produto;

SELECT p.nome_produto
FROM produtos AS p
LEFT JOIN vendas AS v ON p.id_produto = v.id_produto
WHERE v.id_venda IS NULL;

SELECT c.nome, p.nome_produto
FROM clientes AS c
LEFT JOIN vendas AS v ON c.id_cliente = v.id_cliente
LEFT JOIN produtos AS p ON v.id_produto = p.id_produto;
```

14.5 Atividade 8.1

```sql
SELECT c.nome, v.id_venda
FROM vendas AS v
RIGHT JOIN clientes AS c ON v.id_cliente = c.id_cliente;

SELECT p.nome_produto, v.id_venda
FROM vendas AS v
RIGHT JOIN produtos AS p ON v.id_produto = p.id_produto;

SELECT c.nome
FROM vendas AS v
RIGHT JOIN clientes AS c ON v.id_cliente = c.id_cliente
WHERE v.id_venda IS NULL;

SELECT p.nome_produto
FROM vendas AS v
RIGHT JOIN produtos AS p ON v.id_produto = p.id_produto
WHERE v.id_venda IS NULL;

SELECT p.nome_produto, v.id_venda
FROM produtos AS p
LEFT JOIN vendas AS v ON p.id_produto = v.id_produto;
```

14.6 Atividade 9.1

Código 46: Junção completa simulada com UNION

```sql
SELECT p.id_produto, p.nome_produto, v.id_venda
FROM produtos AS p
LEFT JOIN vendas AS v ON p.id_produto = v.id_produto

UNION

SELECT p.id_produto, p.nome_produto, v.id_venda
FROM produtos AS p
RIGHT JOIN vendas AS v ON p.id_produto = v.id_produto;
```

Código 47: Junção completa simulada com UNION ALL

```sql
SELECT p.id_produto, p.nome_produto, v.id_venda
FROM produtos AS p
LEFT JOIN vendas AS v ON p.id_produto = v.id_produto

UNION ALL

SELECT p.id_produto, p.nome_produto, v.id_venda
FROM produtos AS p
RIGHT JOIN vendas AS v ON p.id_produto = v.id_produto
WHERE p.id_produto IS NULL;
```

O MySQL não implementa a sintaxe `FULL OUTER JOIN`. A chave estrangeira impede vendas sem produto existente. `UNION` elimina duplicatas; `UNION ALL` preserva todas as linhas recebidas.

14.7 Atividade 10.1

Consulta	Quantidade esperada
`produtos INNER JOIN vendas`	18 linhas
`produtos LEFT JOIN vendas`	20 linhas
`produtos RIGHT JOIN vendas`	18 linhas
`vendas RIGHT JOIN produtos`	20 linhas

As 20 linhas das junções que preservam produtos correspondem a 18 vendas mais 2 produtos sem venda.

14.8 Atividade 11.1

```sql
SELECT c.id_cliente, c.nome, COUNT(v.id_venda) AS compras
FROM clientes AS c
LEFT JOIN vendas AS v ON c.id_cliente = v.id_cliente
GROUP BY c.id_cliente, c.nome;

SELECT c.id_cliente, c.nome,
       COALESCE(SUM(v.quantidade * v.valor_unitario), 0) AS total
FROM clientes AS c
LEFT JOIN vendas AS v ON c.id_cliente = v.id_cliente
GROUP BY c.id_cliente, c.nome;

SELECT p.id_produto, p.nome_produto,
       COALESCE(SUM(v.quantidade), 0) AS unidades
FROM produtos AS p
LEFT JOIN vendas AS v ON p.id_produto = v.id_produto
GROUP BY p.id_produto, p.nome_produto;

SELECT p.categoria,
       COALESCE(SUM(v.quantidade * v.valor_unitario), 0) AS total
FROM produtos AS p
LEFT JOIN vendas AS v ON p.id_produto = v.id_produto
GROUP BY p.categoria;

SELECT c.cidade, COUNT(v.id_venda) AS compras
FROM clientes AS c
LEFT JOIN vendas AS v ON c.id_cliente = v.id_cliente
GROUP BY c.cidade;

SELECT c.id_cliente, c.nome, COUNT(v.id_venda) AS compras
FROM clientes AS c
INNER JOIN vendas AS v ON c.id_cliente = v.id_cliente
```

```sql
GROUP BY c.id_cliente, c.nome
HAVING COUNT(v.id_venda) >= 2;

SELECT p.id_produto, p.nome_produto,
       SUM(v.quantidade * v.valor_unitario) AS faturamento
FROM produtos AS p
INNER JOIN vendas AS v ON p.id_produto = v.id_produto
GROUP BY p.id_produto, p.nome_produto
HAVING SUM(v.quantidade * v.valor_unitario) > 1000;
```

14.9 Atividade 12.1

a) O nome se repete porque cada venda correspondente gera uma nova linha.

b) `COUNT(*)` conta também a linha preservada com valores nulos; `COUNT(v.id_venda)` conta somente vendas existentes.

c) Um filtro sobre colunas da tabela direita remove linhas cujo valor é `NULL`; quando necessário, coloque o filtro no `ON`.

d) O faturamento histórico usa `vendas.valor_unitario`.

e) A tabela `vendas` contém as duas chaves estrangeiras que representam a compra real de um produto por um cliente.

14.10 Revisão 13.2 — INNER JOIN

```sql
SELECT c.nome, v.data_venda, v.forma_pagamento
FROM vendas AS v
INNER JOIN clientes AS c ON v.id_cliente = c.id_cliente;

SELECT p.nome_produto, p.categoria, v.quantidade,
       v.quantidade * v.valor_unitario AS total
FROM vendas AS v
INNER JOIN produtos AS p ON v.id_produto = p.id_produto;

SELECT c.nome, c.cidade, p.nome_produto, v.data_venda
FROM vendas AS v
INNER JOIN clientes AS c ON v.id_cliente = c.id_cliente
INNER JOIN produtos AS p ON v.id_produto = p.id_produto;

SELECT c.nome, c.estado, v.id_venda
FROM vendas AS v
INNER JOIN clientes AS c ON v.id_cliente = c.id_cliente
WHERE c.estado = 'MG';

SELECT c.nome, p.nome_produto, v.forma_pagamento
FROM vendas AS v
INNER JOIN clientes AS c ON v.id_cliente = c.id_cliente
INNER JOIN produtos AS p ON v.id_produto = p.id_produto
```

```sql
WHERE p.categoria = 'Informática'
  AND v.forma_pagamento = 'Pix';
```

14.11 Revisão 13.3 — junções externas

```sql
SELECT c.nome, v.id_venda
FROM clientes AS c
LEFT JOIN vendas AS v ON c.id_cliente = v.id_cliente;

SELECT c.nome
FROM clientes AS c
LEFT JOIN vendas AS v ON c.id_cliente = v.id_cliente
WHERE v.id_venda IS NULL;

SELECT p.nome_produto, v.id_venda
FROM produtos AS p
LEFT JOIN vendas AS v ON p.id_produto = v.id_produto;

SELECT p.nome_produto
FROM produtos AS p
LEFT JOIN vendas AS v ON p.id_produto = v.id_produto
WHERE v.id_venda IS NULL;

SELECT p.nome_produto, v.id_venda
FROM vendas AS v
RIGHT JOIN produtos AS p ON v.id_produto = p.id_produto;

SELECT p.nome_produto, v.id_venda
FROM produtos AS p
LEFT JOIN vendas AS v ON p.id_produto = v.id_produto
UNION
SELECT p.nome_produto, v.id_venda
FROM produtos AS p
RIGHT JOIN vendas AS v ON p.id_produto = v.id_produto;
```

14.12 Revisão 13.4 — relatórios gerenciais

```sql
SELECT c.id_cliente, c.nome, COUNT(v.id_venda) AS compras
FROM clientes AS c
LEFT JOIN vendas AS v ON c.id_cliente = v.id_cliente
GROUP BY c.id_cliente, c.nome;

SELECT c.id_cliente, c.nome,
       COALESCE(SUM(v.quantidade * v.valor_unitario), 0) AS total
FROM clientes AS c
LEFT JOIN vendas AS v ON c.id_cliente = v.id_cliente
GROUP BY c.id_cliente, c.nome;

SELECT p.id_produto, p.nome_produto,
```

```sql
       COALESCE(SUM(v.quantidade), 0) AS unidades
FROM produtos AS p
LEFT JOIN vendas AS v ON p.id_produto = v.id_produto
GROUP BY p.id_produto, p.nome_produto;

SELECT p.categoria,
       COALESCE(SUM(v.quantidade * v.valor_unitario), 0) AS total
FROM produtos AS p
LEFT JOIN vendas AS v ON p.id_produto = v.id_produto
GROUP BY p.categoria;

SELECT c.estado,
       SUM(v.quantidade * v.valor_unitario) AS faturamento
FROM vendas AS v
INNER JOIN clientes AS c ON v.id_cliente = c.id_cliente
GROUP BY c.estado;

SELECT c.id_cliente, c.nome, COUNT(v.id_venda) AS compras
FROM clientes AS c
INNER JOIN vendas AS v ON c.id_cliente = v.id_cliente
GROUP BY c.id_cliente, c.nome
HAVING COUNT(v.id_venda) >= 2;

SELECT p.nome_produto, p.estoque
FROM produtos AS p
LEFT JOIN vendas AS v ON p.id_produto = v.id_produto
WHERE v.id_venda IS NULL;

SELECT DATE_FORMAT(v.data_venda, '%Y-%m') AS mes,
       SUM(v.quantidade * v.valor_unitario) AS faturamento
FROM vendas AS v
INNER JOIN produtos AS p ON v.id_produto = p.id_produto
WHERE p.categoria = 'Informática'
GROUP BY DATE_FORMAT(v.data_venda, '%Y-%m')
ORDER BY mes;
```

15 Checklist do estudante

Marque cada objetivo quando conseguir explicá-lo e construir um exemplo próprio.

- ○ Identifico as chaves primárias de clientes, produtos e vendas.
- ○ Identifico as chaves estrangeiras presentes em vendas.
- ○ Explico o relacionamento de um para muitos.
- ○ Uso aliases para deixar consultas com várias tabelas mais legíveis.
- ○ Escrevo a condição correta de relacionamento com `ON`.
- ○ Distingo `INNER JOIN`, `LEFT JOIN` e `RIGHT JOIN`.

- Identifico clientes sem compras usando junção externa.
- Identifico produtos nunca vendidos.
- Combino as três tabelas em uma mesma consulta.
- Explico por que aparecem valores `NULL` em junções externas.
- Simulo uma junção completa no MySQL com `UNION`.
- Uso `COUNT(v.id_venda)` para contabilizar compras corretamente.
- Uso `COALESCE` para exibir zero quando não há correspondência.
- Sei quando um filtro deve ficar no `ON` e quando deve ficar no `WHERE`.
- Produzo relatórios agregados por cliente, produto, categoria e estado.

Próxima etapa sugerida

Subconsultas, consultas correlacionadas, views, índices, planos de execução e funções de janela aplicadas a relatórios com múltiplas tabelas.